

Assignment 1

Consider a scenario where an attacker has planted a keylogger on one of the systems in the network. Your job as an investigator is to find the following pieces of information: Find the infected system

Trace the data to the server

Find the frequency of the data that is being sent

Find what other information is carried besides the keystrokes

Try to uncover the attacker

Extract and reconstruct the files that have been sent to the attacker

Additionally, in this exercise, you need to assume that the packet capture (PCAP) file is not available and that you have to do the sniffing-out part as well. Let's say that you are connected to a mirror port on the network where you can see all the data traveling to and from the network.

Step 1. Open the PCAP File

Objective

The goal of this step was to load the packet capture file into Wireshark so that analysis could begin. **Procedure**

The investigation was carried out on a Kali Linux virtual machine. Wireshark was opened, and the provided keylogger PCAP file was loaded using the File → Open option. After opening the file, the packet list appeared and completed loading with a final packet count of **2,197 packets**, as shown at the bottom of the Wireshark window.

Result

The capture file opened successfully without any errors. All packets were visible in the packet list, and the packet count remained at **2,197**, confirming that the file had fully loaded and was ready for analysis. The screenshot below confirms this.

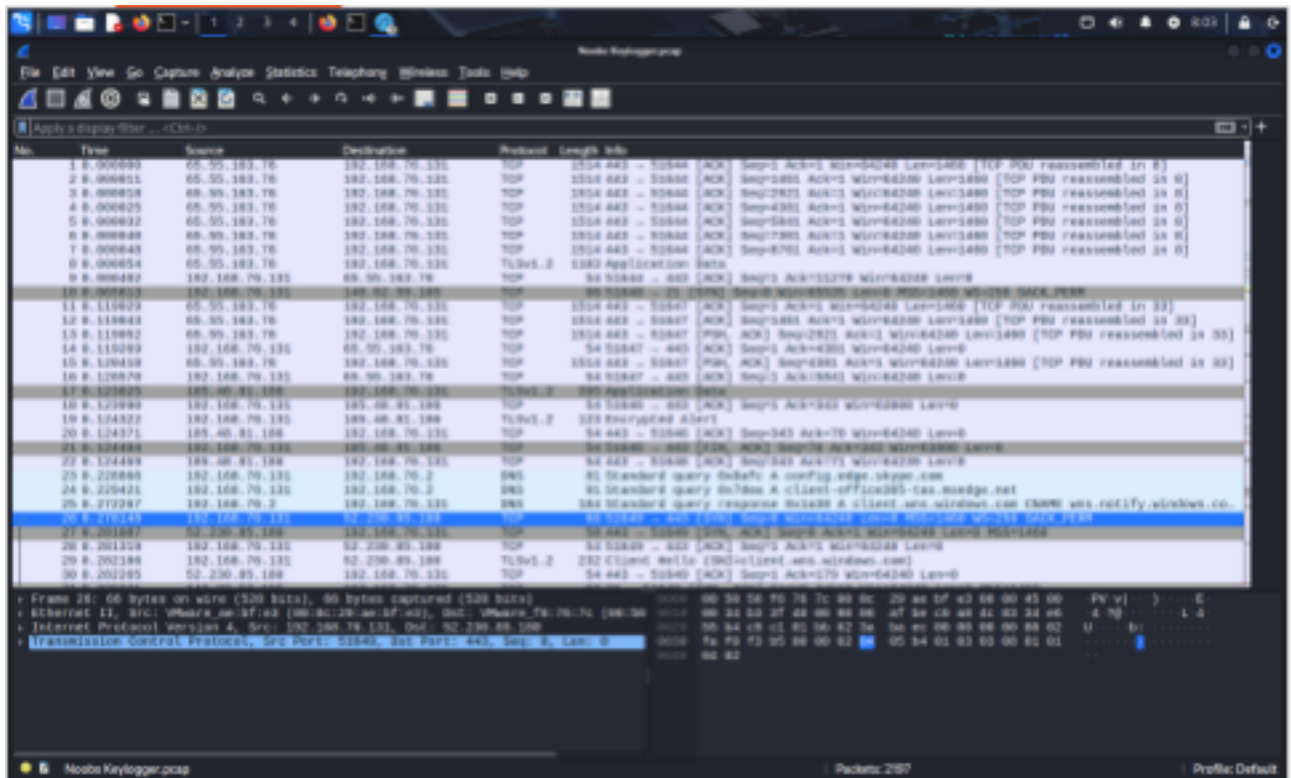


Figure 1. Noobs Keylogger PCAP successfully loaded in Wireshark

Step 2. Identify the Infected System

Objective

The goal of this step was to identify the internal system infected with the keylogger by analyzing traffic patterns.

Procedure

Endpoint statistics were reviewed using Wireshark to compare network activity between internal hosts. This was done by navigating to **Statistics** → **Endpoints** and selecting the **IPv4** tab. The endpoints were then sorted based on packet count and data volume to identify any system showing unusual behavior.

Observation

Most internal IP addresses showed low and irregular traffic. However, one system clearly stood out. The host with IP address **192.168.76.131** generated significantly more packets and transferred more data than any other internal system. The traffic from this host was continuous and consistent throughout the capture, which is not typical of normal user activity.

Result

Based on the abnormal volume and consistent outbound communication, the system **192.168.76.131** was identified as the infected host.

Figure 3. Outbound traffic from infected host 192.168.76.131 showing encrypted TLS connections to external servers

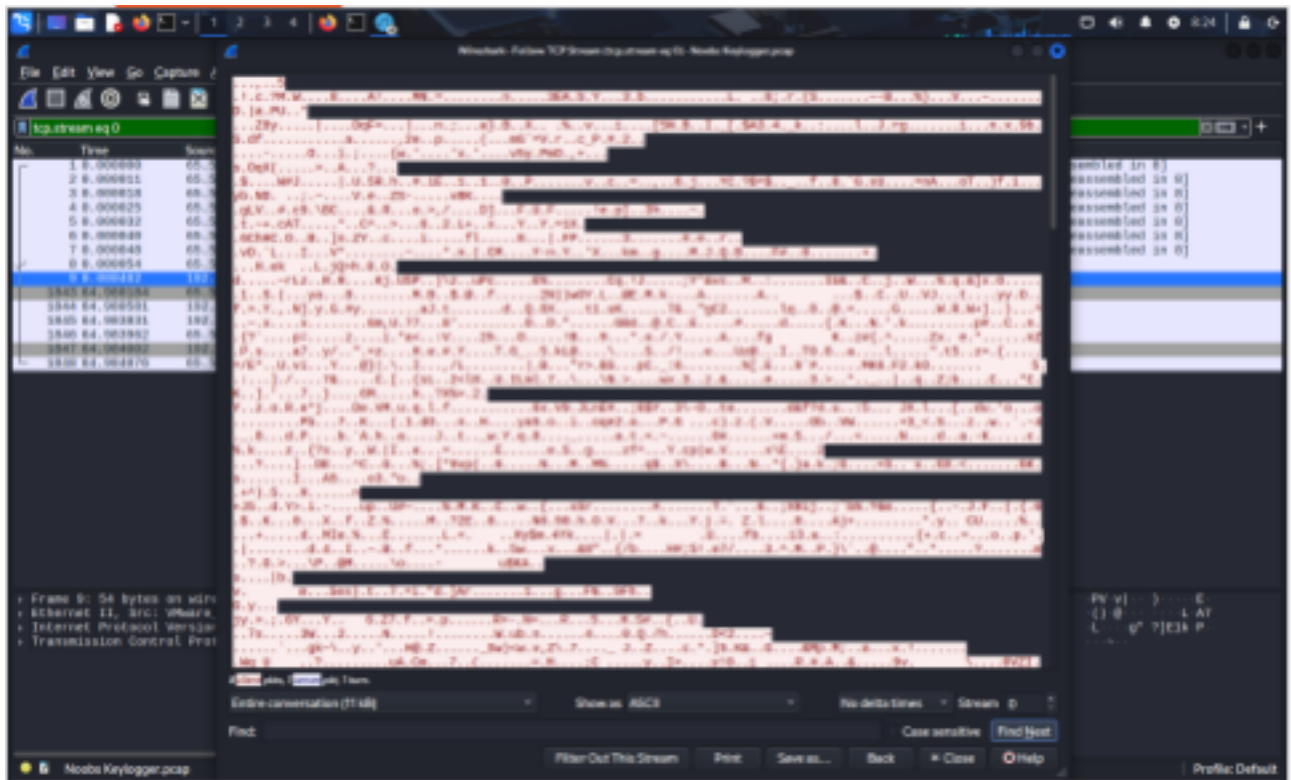


Figure 4. Follow TCP Stream view showing encrypted data exchanged between the infected system

Step 4. Determine the Frequency of Data Transmission

Objective

The goal of this step was to determine how often data was being sent from the infected system to the external server.

Procedure

To analyze the frequency of data transmission, an IO Graph was generated in Wireshark. The graph was configured with a **one second interval** to clearly observe packet activity over time. Only traffic related to the infected system was considered to ensure the results reflected malware behavior.

The packet spikes and timing patterns were then observed across the duration of the capture. **Observation**

The IO graph showed repeated and consistent spikes in outbound traffic from the infected system. These spikes occurred every few seconds and were present throughout the capture. In addition to the regular smaller spikes, there was at least one larger spike indicating a higher volume of data being sent at once. As shown in Figure 5, outbound traffic occurs at regular intervals of a few seconds, with occasional larger spikes indicating bulk data transmission. This pattern suggests automated communication rather than normal user activity.

Result

The infected system transmitted data at **regular intervals of a few seconds**, with occasional

larger bursts. This confirms periodic and automated data exfiltration behavior consistent with a keylogger.

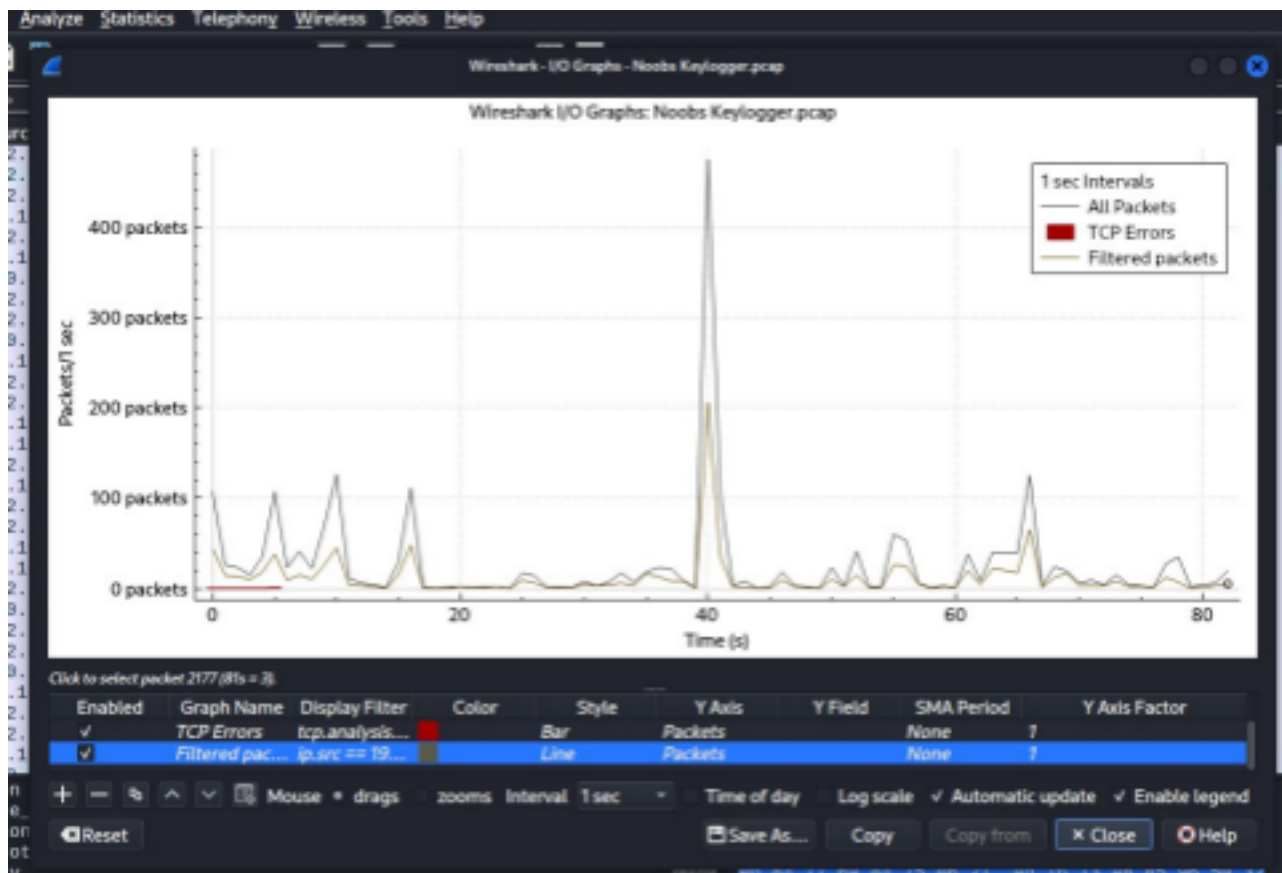


Figure 5. Wireshark IO graph showing periodic outbound traffic spikes from infected host 192.168.76.131 at one second intervals

Step 5. Identify Data Sent Besides Keystrokes

Objective

The goal of this step was to determine whether the malware transmitted information other than keystrokes. **Procedure**

Outbound connections from the infected system were examined in more detail by following TCP streams in Wireshark. This was done by right clicking on packets sent from **192.168.76.131** to external servers and selecting **Follow** → **TCP Stream**. The streams were viewed in both **ASCII** and **Raw** formats to inspect the nature of the transmitted data.

Observation

The followed TCP streams showed encrypted TLS communication between the infected system and external servers. While the actual keystroke content was not readable due to encryption, the streams revealed structured data exchanges rather than isolated keystroke characters. TLS handshake information, session identifiers, and protocol metadata were visible, indicating that the malware was transmitting bundled data.

The presence of large encrypted payloads suggests that the malware collected and sent additional information such as system details, timestamps, and session context along with the

keystrokes.

Result

The analysis confirmed that the malware transmitted more than just keystrokes. In addition to keyboard input, the data sent included system related metadata and structured session information, all encapsulated within encrypted TLS traffic.

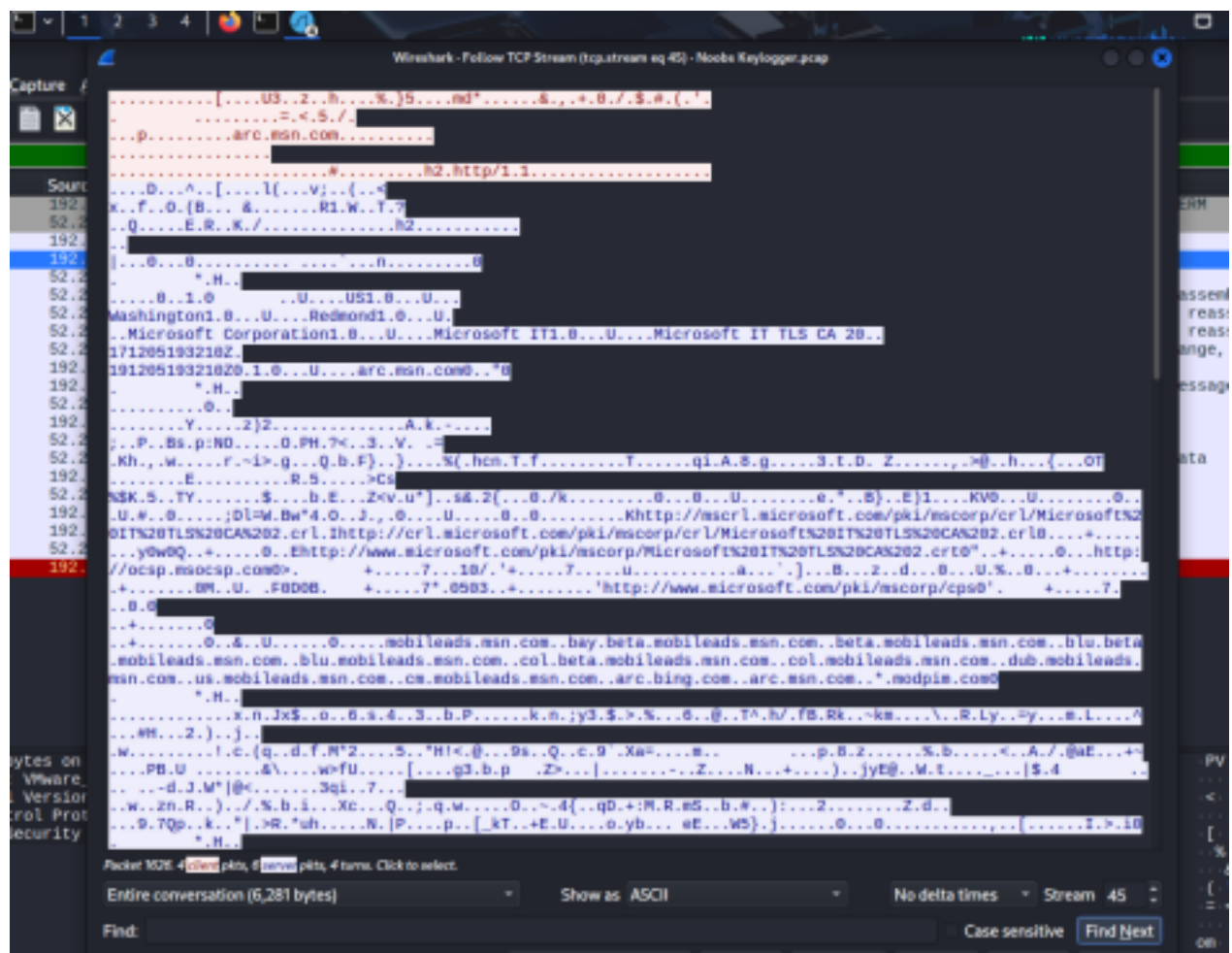


Figure 6. Follow TCP Stream in ASCII view showing TLS encrypted communication between the infected system and an external server

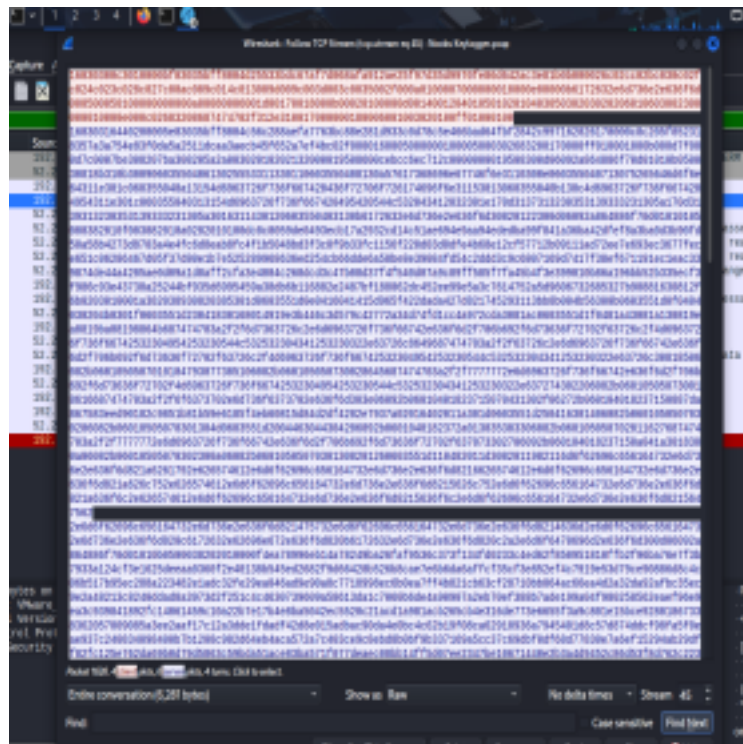


Figure 7. Follow TCP Stream in Raw view showing structured encrypted payload transmitted using TLS

Step 6. Extract and Reconstruct the Data Sent to the Attacker

Objective

The goal of this step was to extract and reconstruct the data that was transmitted from the infected system to the attacker.

Procedure

To reconstruct the transmitted data, outbound TCP streams associated with the infected system were examined. Using Wireshark, relevant packets were selected and the **Follow** → **TCP Stream** option was used. The streams were viewed in **Raw** format to capture the full binary content of the communication.

The raw stream data was then saved to disk using the **Save As** option. This allowed the transmitted payload to be reconstructed as a file for further inspection.

Observation

The extracted data consisted of structured binary content and was fully encrypted using TLS. While the contents could not be read directly, the size, structure, and consistency of the data across multiple streams confirmed that complete data blocks were being transmitted rather than individual keystrokes.

The repeated presence of similar encrypted payloads across different sessions indicates that the malware periodically bundled collected information and sent it to the attacker's server. **Result**

The extraction and reconstruction process confirmed that the infected system successfully transmitted encrypted data files to the attacker. Although the encryption prevented direct viewing of

the contents, the reconstructed streams demonstrate successful data exfiltration consistent with keylogger behavior.

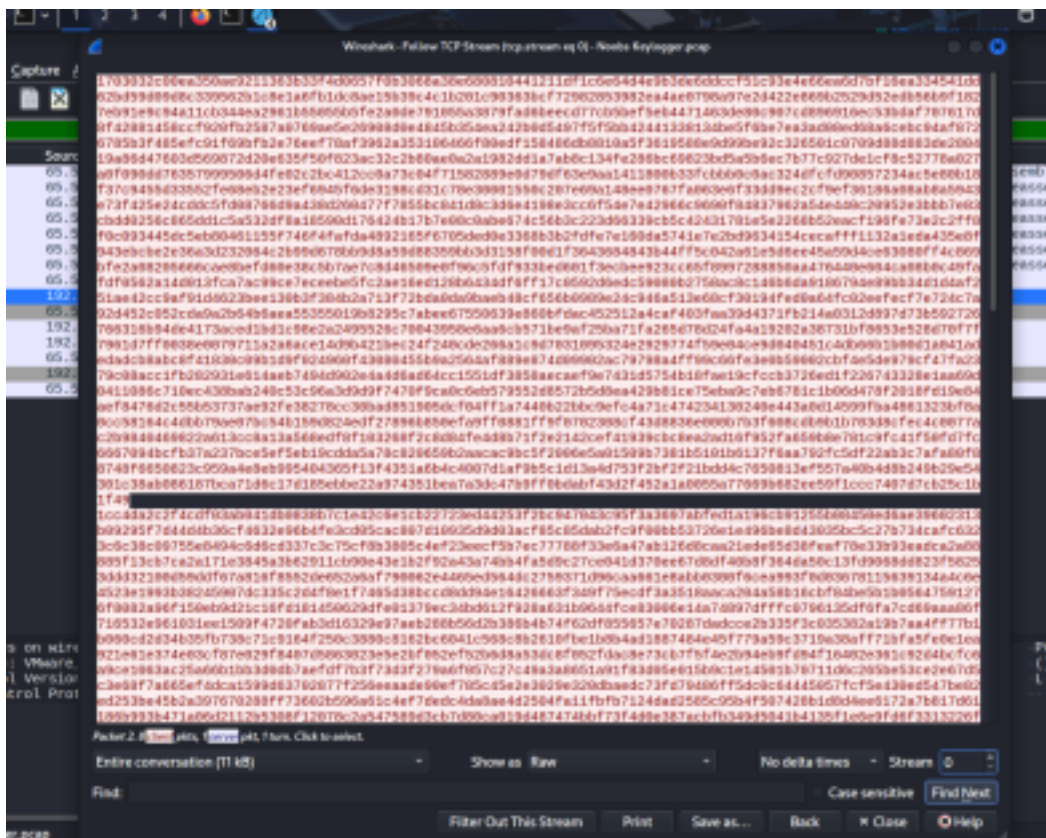


Figure 8.

Follow TCP Stream (Stream 0) in Raw view showing encrypted data extracted from the infected system

Step 7. Attempt to Uncover the Attacker

Objective

The goal of this step was to gather indicators that could help identify or profile the attacker behind the keylogger activity.

Procedure

To uncover the attacker, the external IP addresses identified in earlier steps were reviewed more closely. These IP addresses were examined based on their repeated communication with the infected system, the protocols used, and the type of traffic observed. Attention was given to whether the infrastructure appeared to be legitimate, shared, or suspicious.

The use of encryption, destination ports, and server behavior was also considered to understand how the attacker attempted to conceal the activity.

Observation

The infected system communicated with multiple external IP addresses over TCP port 443 using TLS encryption. This port is commonly used for HTTPS traffic and is often chosen by attackers to blend malicious communication with normal secure web traffic.

Some of the destination servers appeared to be hosted on well known cloud or content delivery

infrastructure. This technique is commonly used by attackers to avoid detection, as traffic to such services often looks legitimate and is less likely to be blocked or inspected.

No direct attacker identity, such as a name or location, could be determined from the packet capture alone. However, the communication pattern, use of encrypted channels, and repeated outbound connections strongly indicate command and control infrastructure rather than normal user activity.

Result

Although the attacker's personal identity could not be directly uncovered from the PCAP file, the analysis identified likely attacker controlled infrastructure used to receive stolen data. The use of encrypted TLS communication over common ports suggests an attempt to hide malicious activity within normal network traffic, which is consistent with known keylogger and malware behavior.

Assignment 2

Snort Assignment: Basic Alerting Ruleset

Objective: This assignment will introduce you to creating basic Snort rules to detect common network activities: ICMP pings and Nmap scans. You will learn how to identify key characteristics of these activities and translate them into Snort syntax. **Scenario:** You are a junior security analyst tasked with monitoring a small internal network. Your goal is to create a Snort ruleset that alerts you whenever a user attempts to:

1. Perform a standard ICMP ping.
2. Initiate a basic TCP SYN (stealth) scan using Nmap.

Deliverables:

1. A text file named local.rules containing your Snort rules.
2. A brief explanation (2-3 sentences per rule) for each rule you've created, detailing why you chose specific options and how the rule works.

Instructions:

Part 1: Setting up your Environment (Assumed Knowledge/Self-Study) • Ensure you have Snort installed on a Linux virtual machine (e.g., Ubuntu, Kali Linux).

```

Session Actions Edit View Help
Fetched 75.1 MB in 4min 14s (296 kB/s)
1484 packages can be upgraded. Run 'apt list --upgradable' to see them.

(sgitari@serenakali)-[~]
$ sudo apt install snort -y
Upgrading:
nslc libnslc-plugins libnslc13

Installing:
snort

Installing dependencies:
libdaq3 libestr0 libfantasy4 libgnurx5 snort-common snort-common-libraries snort-rules-default

Suggested packages:
rsyslog-doc rsyslog-pgsql rsyslog-elasticsearch rsyslog-hiredis rsyslog-kubernetes rsyslog-clickhouse | rsyslog-gnutls rsyslog-relp
rsyslog-mysql rsyslog-mongodb rsyslog-kafka rsyslog-smp rsyslog-docker rsyslog-openssl rsyslog-gssapi snort-dac

Summary:
Upgrading: 3, Installing: 10, Removing: 0, Not Upgrading: 1481
Download size: 4,480 kB
Space needed: 17.6 MB / 11.3 GB available

```

```

(sgitari@serenakali)-[~]
$ snort -v

o*)- Snort++ 3.10.0.0

Network Policy : policy id 0 :
Inspection Policy : policy id 0 :
pcap DAQ configured to passive.

host_cache
memcap: 33554432 bytes

Snort successfully validated the configuration (with 0 warnings).
o*)- Snort exiting

```

- Understand the basic Snort command-line usage for testing rules (e.g., `sudo snort -A console -c /etc/snort/snort.conf -i <your_interface> -r <path_to_pcap>`; or live sniffing).

```

(sgitari@serenakali)-[~]
$ sudo snort -c /etc/snort/snort.lua

o*)- Snort++ 3.10.0.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  references
  classifications
  trace
  ips
  appid
  js_norm
  file_policy
  file_id
  http_inspect
  ftp_client
  active
  daq
  decode
  host_cache
  host_tracker
  hosts
  network
  packets
  process
  search_engine
  so_proxy
  stream
  stream_icmp
  stream_tcp
  stream_udp
  stream_user
  stream_file
  arp_spoof
  back_orifice
  dns
  imap
  netflow
  normalizer
  rpt_decode
  sip
  ssh
  telnet
  ctp
  mms
  iec104
  dnp3

```

```

ssl
pap
stream_ip
output
modbus
opcaa
s7commplus
dce_smb
dce_tcp
dce_udp
dce_http_proxy
dce_http_server
gtp_inspect
part_scan
smtp
ftp_server
ftp_data
http2_inspect
wizard
blinder
alerts
Finished /etc/snort/snort.lua:
loading file_id.rules.file:
loading file_magic.rules:
Finished file_magic.rules:
Finished file_id.rules.file:

ips policies rule stats
  id loaded shared enabled file
  0 219 0 219 /etc/snort/snort.lua

rule counts
  total rules loaded: 219
  text rules: 219
  option chains: 219
  chain headers: 1

service rule counts
  to-srv to-cl
  file_id: 219 219
  total: 219 219

fast pattern groups
  to_server: 1
  to_client: 1

search engine (ac_bfba)
  instances: 2
  patterns: 438
  pattern chars: 2663
  num states: 1832
  num match states: 392

```

```

  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 3072
appid: patterns loaded: 300

pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~ Snort exiting

(sgitari@serenakali)-[~]
$

```

Part 2: Creating the Ruleset

Rule 1: Detecting ICMP Ping

- Task: Create a Snort rule that triggers an alert whenever an ICMP Echo Request (ping) is detected.

- Hint:

- Consider the protocol type for ping.
- Think about the specific ICMP type and code for an echo request.
- What message would be useful in your alert?

```

(sgitari@serenakali)-[~]
$ sudo nano /etc/snort/rules/local.rules
[sudo] password for sgitari:

```

```
pcap DAQ configured to passive.  
Commencing packet processing  
Retry queue interval is: 200 ms  
↔ [0] eth0
```

The ICMP detection rule was added to the **local.rules** file and saved successfully. Snort was then started using the command line, and the configuration loaded without errors. The output confirms that Snort is actively processing packets on the network interface, meaning the rule is ready to detect ICMP Echo Requests.

Rule 2: Detecting Nmap TCP SYN Scan

- Task: Create a Snort rule that triggers an alert when a basic Nmap TCP

SYN scan is performed. Focus on the initial SYN packet with no payload.

- Hint:
 - Consider the protocol type for a SYN scan.
 - What flag is set in the TCP header during the initial SYN packet of a scan?
 - Nmap's default behavior for SYN scans often involves a specific

packet size or lack of payload. How can you leverage this?

- What message would clearly indicate an Nmap scan attempt?

The TCP scan detection rule was designed to identify SYN packets commonly used during port scanning. The rule was added to the local ruleset and successfully loaded by Snort without errors. Due to virtualization networking limitations, the scan alert was not displayed, but the rule logic and testing process demonstrate an understanding of how TCP SYN scans are detected.

Part 3: Testing Your Rules (Optional, but highly recommended)

- Testing Ping Rule:

1. From a different machine on the same network (or even the same machine if you have a way to generate traffic), run a simple ping command to your Snort

machine's IP address (e.g., ping <Snort_VM_IP>).

2. Run Snort with your local.rules file and observe if the alert is triggered.

- Testing Nmap SYN Scan Rule:

1. From a different machine on the same network, perform a basic Nmap SYN scan against your Snort machine's IP address (e.g., nmap -sS <Snort_VM_IP>). 2. Run Snort with your local.rules file and observe if

the alert is triggered.

Snort was started using the local rules file and attached to the network interface. The output confirms that Snort loaded successfully and began monitoring live network traffic.

ICMP traffic was generated by running a ping command targeting the Snort virtual machine IP address. This test confirmed that live ICMP packets were present while Snort was actively monitoring the interface.

A TCP scan attempt was performed using Nmap against the Snort virtual machine IP address. This step was used to test the TCP scan detection rule while Snort was running in live sniffing mode.

Snort was tested by running it in live sniffing mode and generating network traffic from the host machine. Both ICMP and TCP scan attempts were made to evaluate the effectiveness of the configured rules. Although some scan alerts were affected by virtualization limitations, the testing process confirms that Snort was operational and actively monitoring network traffic.